Attacking a Game

Posed by: Tim Wylie

There are several open problems and interesting ways to make progress on what is known about certain games. For several of these, a useful reference may be past course webpages covering many of these topics, which currently has online class notes, useful website links, paper references, and printable versions of some of the games. https://academic.timwylie.com/teaching.html

Complexity & Motion Planning

For any one-player game (a puzzle), the main question is how hard it is to decide if it has a solution. For a two-player game, the main question is whether one of the players (usually the first) has a winning strategy.

For proving hardness, if the question is believed to be in P, simply give an algorithm that shows how to win. If the problem is believed to be NP-hard or PSPACE-hard, you have a few options. Proving the game is NP-hard or PSPACE-hard would be a great feat. The best way to do this is by reducing from a problem we know has that complexity.

- For NP-hardness of 1-player games (puzzles), there are many great example reductions in the free book by Maheshwari and Smid [4]. I would also recommend looking at the paper showing several classic NES games are hard [1].
- If you would like to look at the complexity of a 2-player game, then I recommend using either constraint logic or the motion planning framework. Constraint logic is outlined with examples in Robert Hearn's PhD thesis, which is available online [3].
- The motion planning framework assumes a single agent trying to get to a specific location in a world. If your game can be turned into this, then you should use their gadget framework. There are many related papers, but perhaps the easiest gadget is the locking 2-toggle in [2].

Making an A.I.

Rather than trying to prove the complexity, you could also try to create an AI to learn and play the game well. An entire field of research is dedicated to creating good game-playing AIs. This may seem daunting at first, but there are many good resources to jumpstart any of these efforts - especially in the areas of Q-learning and MCTS (Monte-Carlo Tree Search). Several good resources are listed on my course page that show how to quickly get going with MCTS such as this tutorial using Python (https://jeffbradberry.com/posts/2015/09/intro-to-monte-carlo-tree-search/).

Using an LLM. Another simple A.I. technique is to pass it to an LLM in some manner, either by describing the game and position, or with a screenshot. There are many resources to set this up that can be found online.

Finding a game. There are many games where little is known, but I would suggest researching online for what is known for any game.

References

[1] Greg Aloupis, Erik D. Demaine, Alan Guo, and Giovanni Viglietta. Classic nintendo games are (computationally) hard. *Theoretical Computer Science*, 586:135–160, 2015. Fun with Algorithms, arxiv:1203.1895.

[2] Erik D. Demaine, Dylan H. Hendrickson, and Jayson Lynch. Toward a General Complexity Theory of Motion Planning: Characterizing Which Gadgets Make Games Hard. In *11th Innovations in Theoretical Computer Science Conference (ITCS 2020)*, volume 151 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 62:1–62:42, 2020. arxiv: 1812.03592.

[3] Robert Aubrey Hearn. *Games, Puzzles, and Computation.* PhD thesis, Massachusetts Institute of Technology, Cambridge, MA, USA, 2006. AAI0810083.

[4] Anil Maheshwari and Michiel Smid. *Introduction to Theory of Computation.* Creative Commons, 2017. http://cglab.ca/~michiel/TheoryOfComputation/.